Bilan du projet

SAÉ 3.02 — Développer une application communicante

Groupe	Zero Trust
Membres	Lucas BUCHER — Benjamin DEIBER — Quentin ALTINOK
Enseignant	fdrouhin-uha
Dépôt	https://github.com/debenji68/SAE-3.02

Sommaire

- 1. Bilan des fonctionnalités mises en œuvre
- 2. Conclusion technique et organisationnelle
- 3. Analyse du timing et du respect de la timeline
- Annexe : usage des IA génératives — voir **IAGraphie.pdf**

Rappel du besoin : Le client, responsable des véhicules d'urgence, veut réduire le temps de trajet des pompiers, ambulances et police. La maquette simule un carrefour connecté. Lorsqu'un véhicule d'urgence approche, le feu passe automatiquement au vert pour lui libérer le passage. L'application est répartie sur deux VM Debian : la VM 1 calcule la simulation, enregistre les statistiques dans MariaDB et diffuse l'état par TCP ; la VM 2 affiche la carte et le tableau de bord en PyQt6.

1. Bilan des fonctionnalités mises en œuvre

Revue de chaque fonctionnalité du périmètre fonctionnel du cahier des charges (priorité MoSCoW), puis des choix techniques imposés.

1.1 Périmètre fonctionnel

Fonctionnalité (cahier des charges)	Priorité	État	Ce qui a été fait
Rédaction du cahier des charges	Must	Réalisé	Rendu le 22/09, avant l'échéance du 23/09.
Moteur de simulation (carrefours, densité du trafic)	Must	Partiel	Un carrefour à deux axes, 10 véhicules dont 2 d'urgence. Accélération et freinage progressifs, distance de sécurité de 5 m, lignes d'arrêt à 10 m du centre, cycle automatique vert → orange → rouge. Les virages sont codés et testés (8 combinaisons) mais pas utilisés dans la simulation. La densité est mesurée (vitesse moyenne, nombre de véhicules) mais on ne la fait pas varier, et il n'y a qu'un seul carrefour.
Algorithme d'itinéraire optimal (point A → point B)	Must	Non Réalisé	Le projet s'est recentré sur la priorité au carrefour. Le module GPS (formule de Haversine) est prêt pour calculer des distances réelles entre plusieurs carrefours.
Logique V2I (feux tricolores)	Must	Réalisé	Règle des 50 m : le véhicule d'urgence force son feu au vert et l'axe perpendiculaire au rouge, jusqu'à sa sortie du carrefour. Si deux urgences arrivent en même temps, la première détectée passe.

Logique V2V (alerte des usagers)	Must	Non Réalisé	Les voitures réagissent seulement au feu et au véhicule de devant ; aucune alerte entre véhicules.
Base de données (Stockage des feux, accès, usagers)	Must	Modifié	MariaDB avec deux tables : *evenement* (chaque passage prioritaire, avec distance et horodatage) et *info_traffic* (vitesse moyenne et nombre de véhicules, chaque seconde). Pas de tables feux, de tables accès et usagers : nous avons stocké ce que la simulation produit réellement.
Scripts PowerShell (autorisation d'accès, création des VM)	Should	Modifié	Scripts Bash : installation complète, création de la base, pare-feu ufw, utilisateur MariaDB dédié, recherche automatique du serveur. La création des VM elle-même reste manuelle.
Gestion et structuration du dépôt GitHub	Must	Réalisé	Dépôt structuré par catégorie, il y a une branche par membre en plus de la branche main.On a fait des pull request de fusion, il y a un «. gitignore » et « .gitattributes » et « requirement.txt ». On retrouve également un dossier avec les différents documents de type livrable.
Documentation d'installation et d'usage	Should	Modifié	Regroupée dans le README.md comme demander dans les consignes au lieu de « docs/installation.md » et « docs/usage.md ». Installation pas à pas, commandes, exemples, dépannage.
Cartographie logicielle / IA-graphie	Should	Réalisé	Schéma d'architecture dans le README, carte du carrefour en coordonnées GPS dans l'interface, IA-graphie dans IAGraphie.pdf.
Comptes rendus hebdomadaires	Must	Partiel	CR1 à CR5 sont dans le dépôt.
Livrable finances	Should	Non réaliser	C’est une partie que nous avions définis comme optionnel et qui n’a donc pas été mit en priorité dans la conception de ce dernier.
Document de réponse (docs/reponses.md)	Should	Modifié	Remplacé par le présent Bilan_Projet.pdf comme demander dans les consignes.

1.2 Choix techniques du cahier des charges

Exigence technique	État	Commentaire
Architecture distribuée sur 2 VM Debian	Réalisé	VM 1 : simulation, serveur TCP, MariaDB. VM 2 : interface et client TCP.
Serveur TCP sur le port 5000, état diffusé en JSON	Réalisé	Classe ServeurSimulation : une ligne JSON par état, 10 fois par seconde. Un client à la fois, avec reconnexion sans redémarrer le serveur. La simulation avance tant qu'un client est connecté, et un nouveau scénario démarre quand le précédent est terminé.
Client TCP dans un thread dédié	Réalisé	QThread avec signaux Qt, découpage des messages, reconnexion automatique, arrêt propre à la fermeture.
Interface PyQt6 : carte et tableau de bord	Réalisé	Carte du carrefour (feux, voitures, véhicules d'urgence sur leur voie) et onglet de statistiques.
mysql-connector-python vers MariaDB	Réalisé	Classe StatsDB : enregistrements et lecture des indicateurs.
Conversion GPS → pixels avec le module math	Réalisé	gps_utils.py : mètres ↔ GPS ↔ pixels, et distance de Haversine.

Déploiement automatisé par scripts Bash	Réalisé	setup_vm1_server.sh et setup_vm2_client.sh.
Environnement virtuel venv et requirements.txt	Réalisé	requirements.txt à la racine, les scripts créent le venv et installent les paquets nécessaires à chaque VM.
Python 3.14.7	Réaliser	Les 2 VM tourne sous Debian 13, donc possède la version 3.14.7 de Python

1.3 En résumé

La demande du client qui est que le feu passe au vert à l'approche d'un véhicule d'urgence est en place et est visible en temps réel sur une autre machine avec des statistiques enregistrées. Deux fonctionnalités *Must have* n'ont pas été développées : l'itinéraire du point A au point B et la communication V2V. Nous avons préféré livrer un carrefour complet et fonctionnel plutôt que plusieurs fonctionnalités avec un carrefour qui fonctionne partiellement.

2. Conclusion technique et organisationnelle

2.1 Difficultés techniques rencontrées sous Python

La plupart des difficultés ne venaient pas du code de chaque module, qui fonctionnait seul, mais de l'assemblage des différentes parties.

Difficulté	Ce qui se passait	Comment nous l'avons surmontée
Imports et paquets Python	*from map_canvas import* marchait en lançant le fichier seul mais plantait via main.py (ModuleNotFoundError). Le fichier network/__init__.py importait le serveur, importer le client lançait un serveur sur la VM 2.	Imports relatifs, lancement par *python -m projet_simulation.main*, __init__.py vidé pour que la VM 1 et la VM 2 ne chargent que leurs propres dépendances.
TCP est un flux, pas une suite de messages	Un JSON pouvait arriver couper en deux ou collé au suivant et donc json.*loads* échouait.	Mise en place d'une règle : « une ligne JSON par message » et également une autre qui fait en sorte que le client ne décode que les lignes complètes.
Threads et interface Qt	Lire la base de données à chaque message dans le thread graphique figeait la fenêtre ; fermer la fenêtre pendant que le thread réseau tournait faisait planter Qt.	Client dans un QThread qui communique par signaux ; les indicateurs arrivent dans le JSON et arrêt du thread dans closeEvent.
Feu forcé qui repassait au rouge	Écrire *feu.etat = "vert"* ne remettait à zéro ni la transition orange ni le chrono : le feu forcé repassait à l'orange environ 2 s plus tard.	Méthode *forcer_etat()* dans feuDeCirculation qui annule la transition et relance le chrono.
Base de données	*connect(self.config)* au lieu de *connect(**self.config),* AVG renvoyait NULL sur une table vide ce qui donnait « None km/h ».	Correction de la connexion à la BDD, *COALESCE(AVG(...), 0)*.
Repères de coordonnées	La simulation calcule en mètres (Y vers le haut) alors que l'écran est en pixels (Y vers le bas) : tout s'affichait coller en haut à gauche.	Conversion mètres → GPS → pixels dans gps_utils.py.

Réaliser un socket serveur	Le socket ayant pour rôle serveur se comportait comme un client se qui créait une erreur et ne lançait pas la simulation.	Correction du fichier socket serveur afin qu'il se comporte comme tels.

2.2 Bilan organisationnel

Ce qui a bien marché. Le découpage en sous-parties (models, database, network, gui, utils) a permis de travailler à trois en parallèle, chacun responsable d'une partie. Le dépôt GitHub a servi de regroupement des différentes parties, et les comptes rendus des premières semaines ont fixé l'architecture tôt.

Ce qui nous a coûté du temps. Au CR3, nous avons décidé de développer et tester chaque partie séparément avant de les intégrer. L'idée était bonne pour isoler les erreurs, mais nous ne nous étions pas mis en accord sur le développement des parties : format exact du JSON, noms des signaux, paramètres des fonctions. Résultat : chaque partie marchait seule, mais l'intégration des codes à créer des erreurs, qu'il a fallu corriger dans les fichiers des autres.

Ce que nous ferions autrement.

- Se mettre en accord sur le développement du code et des différents paramètres : un exemple de message JSON, la liste des classes et de leurs méthodes, les noms des signaux.
- Faire tourner une version minimale du projet (un véhicule, un feu, de la VM 1 à la VM 2) dès les premières semaines, puis l'enrichir, au lieu d'intégrer à la fin.
- Après chaque mise à jour, faire un point entre membres avant de pousser sur GitHub.

3. Analyse du timing et du respect de la timeline

Le prévu vient du rétroplanning et du tableau des échéances du cahier des charges. Le réel vient des comptes rendus CR1 à CR4 et de l'historique des commits Git.

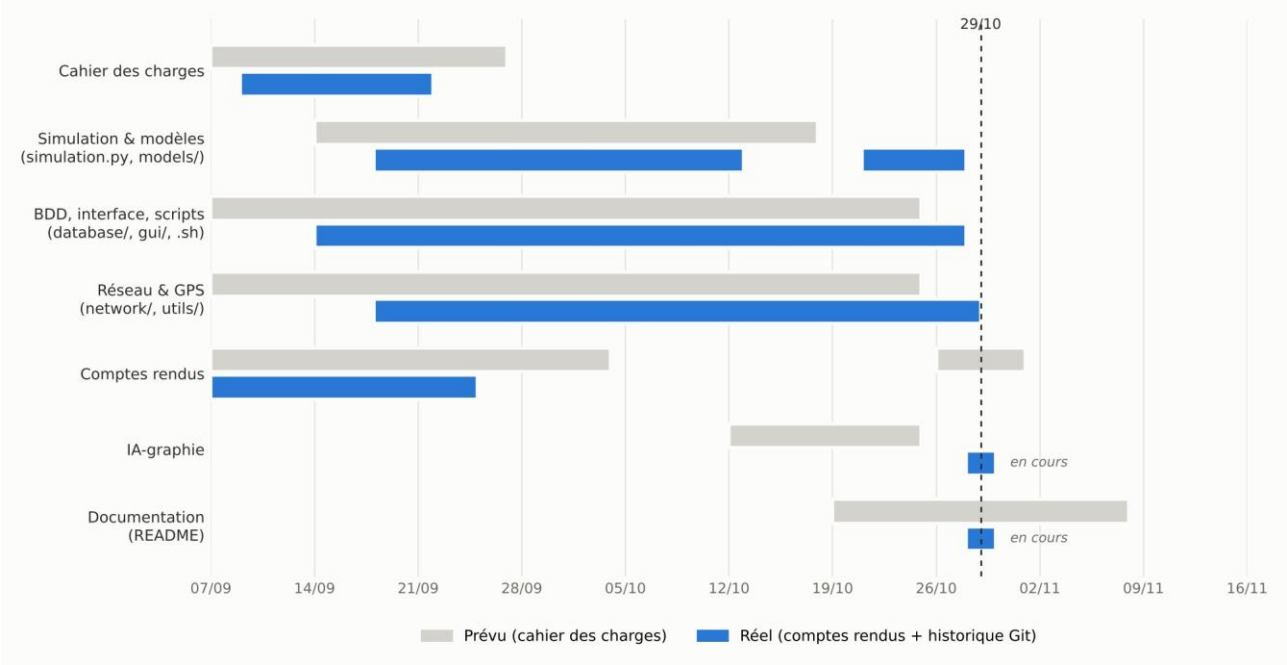


Figure 1 — Planning prévu (gris) et réel (bleu). La ligne pointillée marque la date de ce bilan.

3.1 Jalons

Livrable	Échéance prévue	Réalisé	Écart
Cahier des charges	23/09	22/09	Dans les temps

Simulation et modèles (models/, simulation.py)	15/10	11/10 ; reprise pour l'intégration du 21/10 au 28/10	En avance, puis +13 jours d'intégration
BDD, interface, scripts Bash	25/10	BDD : 20/10, scripts : 22/10, interface intégrée : le 28/10	+3 jours
Réseau et GPS (network/, utils/)	25/10	GPS : 20/10 ; client fonctionnel : 26/10 ; serveur intégré le 28/10	+3 jours
IA-graphie	25/10	13/11	Dans les temps
Livrable	**Échéance prévue**	**Réalisé**	**Écart**
Documentation (README)	15/11	13/11	Dans les temps
Comptes rendus CR1 à CR5	13/09 ; 21/09 ; 04/10 ; 30/10 ; 15/10	Présents dans le dépôt	Dans les temps

3.2 Temps de travail

Le cahier des charges estimait la durée de réalisation de la SAE à un total à **66 heures**. Nous estimons que pour réaliser cette SAE, il nous a fallu environ 50 heures de travail (heures de cours et heures de travailles personnel).

L'activité Git par semaine (tous membres, tous commits confondus, y compris les modifications du README) montre où s'est concentré le développement :

Semaine	**07/09**	**14/09**	**21/09**	**28/09**	**05/10**	**12/10**	**19/10**	**26/10**
Commits	4	3	0	19	3	15	18	12

3.3 Analyse des décalages

- Démarrage tardif du code. Aucun commit de code avant le 30/09 : la semaine du 21/09 n'en compte aucun. Les trois premières semaines ont servi au cadrage et à l'apprentissage des notions nécessaires (sockets, PyQt6, MariaDB), ce que le planning n'avait pas prévu.
- La simulation a été finie en avance (11/10 pour le 15/10), mais elle a dû être restructurée pour être utilisée par le serveur : cette étape n'était pas planifiée.
- Le réseau a été livré en dernier (serveur le 28/10, pour une échéance au 25/10). Il dépend de toutes les autres : le serveur ne peut envoyer la simulation qu'une fois celle-ci intégrable, et le client doit respecter le format attendu par l'interface. Ces dépendances n'apparaissaient pas dans le rétroplanning, où toutes les lignes avançaient en parallèle.
- L'intégration est concentrée sur les dernières semaines, conséquence directe de la décision de développer séparément puis d'intégrer.
- La documentation est en avance parce que l'installation automatisée a imposé de décrire précisément le lancement de l'application dès la fin octobre.

Conclusion sur le planning. Les échéances individuelles ont été globalement tenues, à quelques jours près. Le vrai décalage vient de l'intégration, qui n'avait pas de ligne propre dans le planning alors qu'elle a occupé la fin du projet. Pour un futur projet, il faudrait définir une partie « reconstitution du projet » dans le planning afin de ne pas se faire surprendre sur cette partie en fin de projet qui est importante.